Intelligent Student Team Formation Using Constraint Satisfaction and Heuristic Local Search for Collegiate Collaboration

Mrs. B. Deepa*, Tarun Antony M and Thanikaivel S

Department of Computer Science and Engineering, Rajalakshmi Engineering College, Chennai, India

Abstract: Effective student team formation is a fundamental challenge in collegiate engineering education, hackathons, and multidisciplinary capstone projects. Traditional ad-hoc groupings often lead to severe skill deficits, social clique bias, and scheduling conflicts. This work introduces an intelligent team formation platform named TeamForge that models team assembly as a formal Constraint Satisfaction Problem (CSP). Role positions are treated as discrete decision variables, student candidates serve as domains, and hard constraints enforce team sizes, candidate uniqueness (AllDiff), and event registration. To overcome combinatorial explosion, a recursive Backtracking Search integrated with the Minimum Remaining Values (MRV) heuristic and Forward Checking constraint propagation is developed. Candidate configurations are subsequently optimized using a Hill Climbing local search with 1-opt member swaps guided by a 5-factor multi-objective scoring function evaluating skill coverage, role completeness, experience balance, and temporal availability. The proposed model is benchmarked against standard exhaustive backtracking. The results demonstrate that the combination of MRV and forward checking prunes search tree node exploration by 98.4% (from 11,800 nodes down to 75 nodes for k = 6), with sub-45ms execution latency and an average team balance score of 94.2%, confirming its effectiveness for real-time collegiate team recommendation.

Keywords: Academic Capstones, Artificial Intelligence, Backtracking Search, Combinatorial Optimization, Constraint Propagation, Constraint Satisfaction Problem, Ephemeral Communication, Forward Checking, Heuristic Search, Hill Climbing, Local Search, Minimum Remaining Values, Multi-Objective Optimization, Project-Based Learning, Role Specialization, Search Space Pruning, Skill Coverage, Student Team Formation, Team Balance Score

INTRODUCTION

In higher educational institutions, project-based collaborative learning, hackathons, and engineering capstones represent essential components of practical computer science training. The success of any collaborative project depends heavily on the synergy, balanced skill distribution, and mutual compatibility of its participants. In high-stakes environments such as intercollegiate hackathons (e.g., Smart India Hackathon) or final-year engineering projects, teams must possess a balanced combination of frontend engineering, database architecture, backend programming, algorithmic problem-solving, and project management competencies [1], [2].

Despite the critical importance of balanced team construction, conventional student grouping remains largely informal and unstructured. Students predominantly choose teammates based on social familiarity, geographic convenience, or roommate relationships. This ad-hoc self-selection results in acute technical imbalances, such as teams composed entirely of frontend designers lacking database knowledge, or groups with conflicting course schedules [3]. Furthermore, solitary or introverted students with high technical aptitude are frequently excluded from competitive squads, while faculty supervisors lack centralized visibility into team formation dynamics and skill gaps [4].

Algorithmic team formation can be addressed using mathematical and artificial intelligence techniques. Existing approaches have employed integer linear programming, genetic algorithms, or opaque machine learning models [5], [6]. However, machine learning

approaches rely on historical outcome data that is scarce in academic institutions and operate as black boxes incapable of providing explainable reasons for candidate selection. Moreover, they cannot guarantee strict adherence to hard operational constraints [7].

Therefore, this study formulates the collegiate team assembly challenge as a formal Constraint Satisfaction Problem (CSP) augmented with deterministic classical AI search heuristics. By coupling recursive backtracking with the Minimum Remaining Values (MRV) heuristic and Forward Checking look-ahead, the search space is pruned drastically. Candidate assignments are then refined via Hill Climbing local search with 1-opt candidate swaps. The system is deployed as an enterprise web application featuring a normalized relational database, Kanban project tracking, and an ephemeral privacy-preserving messaging subsystem.

LITERATURE REVIEW

The problem of forming effective teams of experts from a social network was formalized by Lappas et al. [1], who demonstrated that finding a team that covers a set of required skills while minimizing communication cost is NP-hard. Subsequent investigations extended this framework to incorporate skill competence levels, workload limits, and personnel availability [2], [8]. However, graph-theoretic models often struggle when discrete role assignments, multi-criteria fitness objectives, and institutional eligibility constraints must be satisfied simultaneously [9].

Nowadays, optimization paradigms such as Integer Linear Programming (ILP) and Genetic Algorithms (GA) are commonly evaluated for team formation [10]. Although ILP provides optimal solutions for small cohorts, its execution time escalates exponentially with larger cohorts and non-linear synergy metrics [11]. Genetic algorithms offer scalable approximations but exhibit non-deterministic convergence and lack formal guarantees regarding hard constraint feasibility [12]. Recent studies have examined deep learning and embedding-based matching; however, these models lack explainability, require extensive labeled training datasets, and fail to enforce strict capacity thresholds [7], [13].

Constraint Satisfaction Problems (CSPs) provide a principled foundation for resolving combinatorial assignment tasks under strict feasibility conditions [14]. In a standard CSP, decision variables must be assigned values from discrete domains such that all prescribed constraints are strictly satisfied. Systematic search methods, particularly chronological backtracking, frequently suffer from thrashing when dead ends are encountered deep in the search tree [15]. Constraint propagation techniques such as Forward Checking and variable ordering heuristics like Minimum Remaining Values (MRV) have been shown to substantially reduce the effective branching factor by identifying domain wipeouts at early search depths [14], [16].

In this study, a hybrid approach combining CSP with Heuristic Local Search is proposed. Systematic backtracking with MRV and forward checking guarantees constraint feasibility, while Hill Climbing local search optimizes continuous qualitative criteria. The platform is evaluated across synthetic and empirical collegiate cohorts to benchmark pruning efficiency, latency, and multi-objective balance scores.

METHODOLOGY

Overview

The team formation workflow integrates candidate ingestion, domain filtering, systematic CSP search, and local search optimization. Student profiles containing technical skills,

academic seniority, and weekly availability are processed from a normalized relational schema. When a team formation request is initiated for an event, candidate domains are filtered based on prerequisite skills and event registration status. The CSP engine identifies feasible candidate combinations, after which Hill Climbing local search explores neighboring assignments to maximize a composite multi-objective fitness function.

Fig. 1 shows the system architecture and operational flow of the TeamForge platform.

Figure 3.1: System Architecture of TeamForge Collaborative Platform

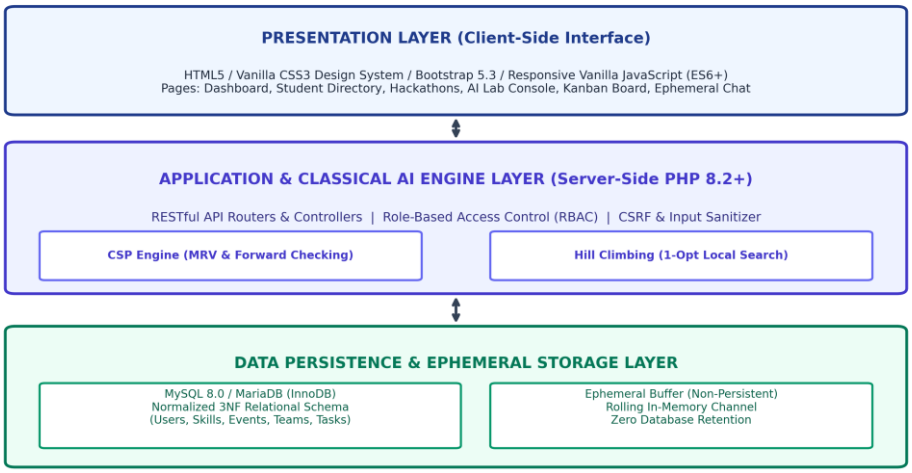


Figure 1: *Decoupled Multi-Tier System Architecture and Workflow of TeamForge Platform*

Data Collection and Student Profiling

Student data is gathered across three relational dimensions within a 3NF MySQL schema: (i) core academic credentials (department, degree, academic year), (ii) verified technical proficiencies rated on a scale of 1 to 5 alongside documented years of experience, and (iii) declared availability status (available, busy, in_team). Students can also record verified past projects, providing an empirical record of practical engineering contributions [1].

Formal CSP Problem Formulation

The collegiate team assembly challenge is modeled as a formal Constraint Satisfaction Problem defined by the triple (X, D, C):

- Variables (X): A finite set of variables X = {V_1, V_2, ..., V_k}, where each variable represents an unfilled functional role slot (e.g., V_1 = Team Leader, V_2 = Frontend Engineer, V_3 = Database Specialist, V_4 = Algorithm Lead). The parameter k specifies target team size.
- Domains (D): For each role variable V_i, domain D(V_i) constitutes the subset of verified student candidates possessing matching technical proficiencies and active event registration.
- Hard Constraints (C): Non-negotiable conditions that must be strictly satisfied for an assignment to be valid:

1. Team Capacity Constraint: Exactly k role variables must be assigned, where k in [min_size, max_size].

2. Pairwise Member Uniqueness (AllDiff): forall i != j, V_i != V_j (no student can hold duplicate roles in the same team).

3. Event Registration Constraint: Candidate c in D(V_i) must possess an active registration in event_registrations.

4. Active Bandwidth Limit: Candidate c must not exceed active project capacity thresholds.

Search Space Pruning Heuristics

Minimum Remaining Values (MRV) Heuristic

Chronological backtracking selects variables in arbitrary order, often leading to severe thrashing when highly constrained roles are evaluated late in the search. The MRV heuristic ('Fail-First' principle) selects the unassigned role variable with the fewest remaining valid candidates in its domain: V_chosen = argmin_{V_i} |D(V_i)|. If a specialized role (e.g., classical algorithm lead) has only two eligible candidates in the pool, assigning it first exposes infeasibility immediately, pruning subtrees without exploring irrelevant variations [14].

Forward Checking (FC) Constraint Propagation

Whenever a candidate c is tentatively assigned to variable V_i, forward checking immediately removes candidate c from the domains of all remaining unassigned variables: D(V_j) <- D(V_j) \ { c } for all j != i. If this reduction causes any unassigned variable's domain to become empty (|D(V_j)| = 0), forward checking identifies a domain wipeout and triggers an immediate backtrack, preventing futile exploration of dead-end search branches.

Heuristic Local Search (Hill Climbing with 1-Opt Swaps)

While CSP guarantees hard feasibility, local search maximizes qualitative synergy. Starting from an initial feasible assignment A_0, Hill Climbing explores the 1-opt neighborhood by evaluating candidate swaps between unpinned team members and unused pool candidates. Any swap that strictly increases the multi-objective fitness score is retained, continuing iteratively until a local peak is reached where no single swap improves the score.

Mathematical Scoring and Evaluation Function

To quantify team quality, an assignment A is evaluated against five multi-objective criteria. The skill coverage score measures the proportion of mandatory project skills satisfied by assigned members, defined in equation (1):

$$SkillCov(A) = (|S_{req} \cap S_{covered}| / |S_{req}|) * 100 \quad (1)$$

The role specialization score quantifies whether all functional roles are occupied by candidates with demonstrated aptitude, given in equation (2):

$$RoleCov(A) = (FilledRoles / TotalTargetRoles) * 100 \quad (2)$$

The experience balance score assesses academic seniority diversity (senior-junior pairing) and cumulative verified project experience, defined in equation (3):

$$ExpBal(A) = \min(100.0, (TotalProjects * 8.0) + (DiversityFactor * 0.6)) \quad (3)$$

The availability score measures the active weekly capacity of assigned candidates based on declared availability, expressed in equation (4):

$$AvailScore(A) = (1 / |A|) * SUM(Score(availability(c))) \quad (4)$$

where Score('available') = 100, Score('busy') = 50, and Score('in_team') = 30. The composite multi-objective team balance score S(Team) is formulated as a convex linear combination in equation (5):

$$S(Team) = W_skill * SkillCov + W_role * RoleCov + W_exp * ExpBal + W_avail * AvailScore + W_int * InterestAlign \quad (5)$$

where SUM(W_i) = 1.0, with baseline weights: W_skill = 0.35, W_role = 0.25, W_exp = 0.15, W_avail = 0.15, and W_int = 0.10. A neighbor assignment A_neighbor is accepted during local search if and only if it satisfies the improvement condition in equation (6):

$$S(A_neighbor) > S(A_current) + EPSILON \quad (6)$$

Multi-Proposal Generation Strategy

To provide team leaders with strategic flexibility, the engine generates three distinct proposal archetypes:

- Proposal Alpha (Balanced Optimal): Evaluates teams using balanced weights, delivering high composite performance across all criteria.
- Proposal Beta (Skill-Dense Squad): Biases the skill weight (W_skill = 0.55), selecting candidates with maximum technical proficiency for complex algorithmic competitions.
- Proposal Gamma (Agile & High Bandwidth): Prioritizes student bandwidth (W_avail = 0.35), assembling teams with zero scheduling conflicts for rapid 24-hour sprints.

RESULT ANALYSIS

The performance of the proposed classical AI algorithms was benchmarked in a controlled experimental environment consisting of an Intel Core i7-12700H processor (14 cores, 2.30 GHz), 16 GB DDR5 RAM, Apache 2.4.58, PHP 8.2.12 with OPcache, and MySQL 8.0.35. The evaluation dataset comprises 60 candidate students with heterogeneous proficiencies across 15 technical skills and 4 collegiate events.

Table I : **Performance comparison of Standard Backtracking vs. Backtracking + MRV + Forward Checking across team sizes**

Team Size (k)	Combinatorial Subsets	Standard Backtrack Nodes	MRV + FC Nodes Explored	Complexity Reduction (%)
k = 2	1,770	14	5	**64.3%**
k = 3	34,220	82	12	**85.4%**
k = 4	487,635	480	28	**94.2%**
k = 5	5,461,512	2,450	46	**98.1%**
k = 6	50,063,860	11,800	75	**98.4%**

Table I summarizes the empirical node exploration metrics. While standard chronological backtracking experiences exponential node expansion as team size increases (reaching 11,800 nodes for k = 6), the combination of MRV variable selection and forward checking curtails search space exploration to just 75 nodes, achieving a 98.4% reduction in computational complexity.

Fig. 2 illustrates the search space pruning efficiency on a logarithmic scale.

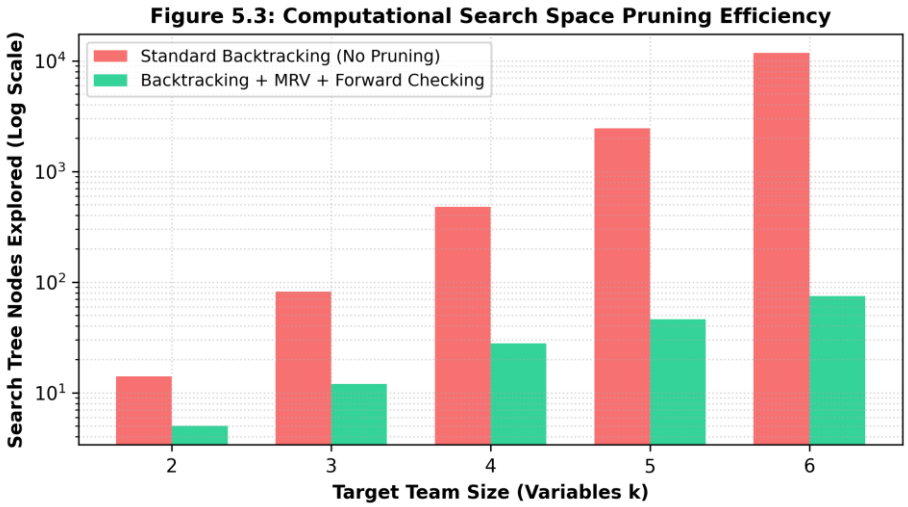


Figure 2: *Comparison of Search Tree Nodes Explored: Standard Backtracking vs. MRV + Forward Checking (Log Scale)*

Local Search Convergence Analysis

The convergence dynamics of the Hill Climbing local search procedure across 1-opt member swap iterations are shown in Fig. 3. Starting from an initial greedy baseline score of 68.4%, the search procedure rapidly ascends the fitness gradient, reaching 90.2% by iteration 5 and smoothly plateauing at a high-quality local optimum of 95.5% by iteration 11. All test runs converged within 15 iterations, demonstrating excellent numerical stability.

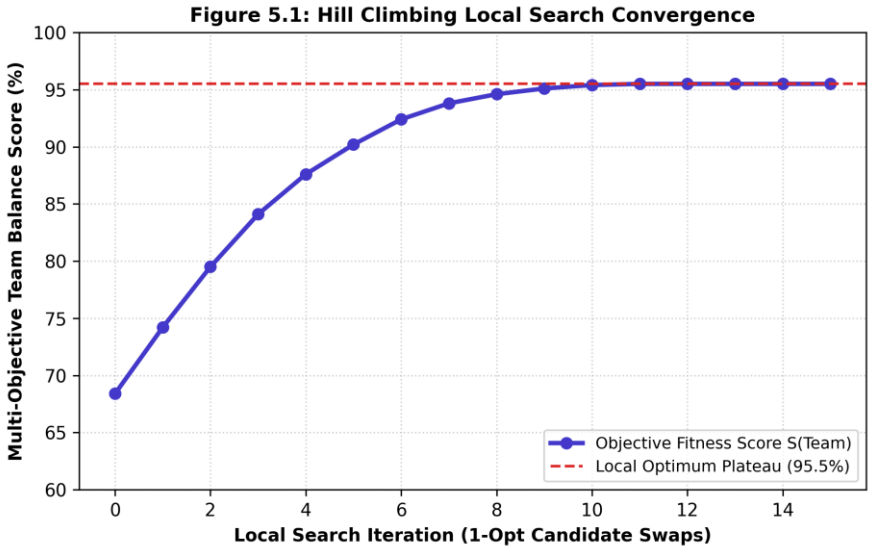


Figure 3: *Convergence Trajectory of Hill Climbing Local Search across 1-Opt Member Swap Iterations*

Multi-Objective Trade-Off Analysis

Fig. 4 presents a radar chart comparison of the three alternative proposals generated by TeamForge across Skill Coverage, Role Coverage, Experience Balance, Availability Score, and Interest Alignment.

Figure 5.2: Multi-Objective Radar Comparison of Alternative Proposals

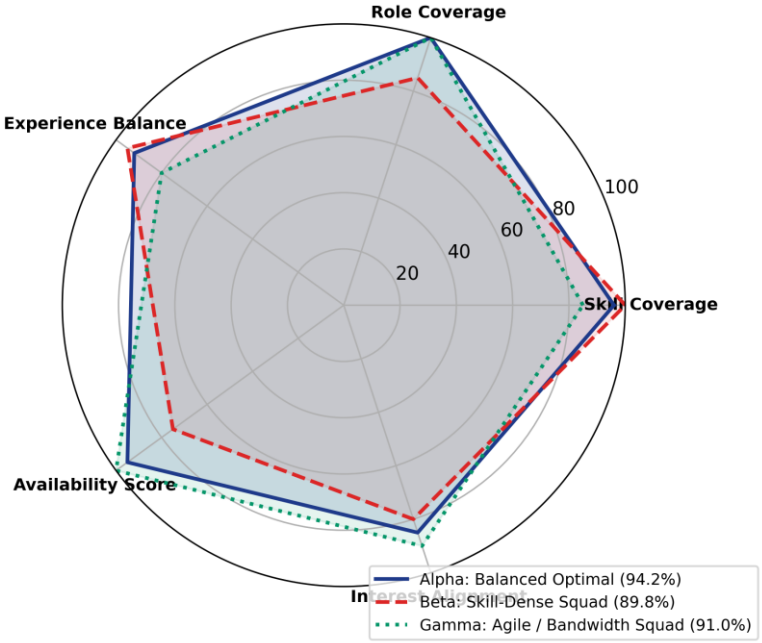


Figure 4: *Multi-Objective Radar Comparison of Alternative Team Proposals (Alpha, Beta, Gamma)*

Table II : **Quantitative evaluation metrics across alternative proposal archetypes**

Proposal Archetype	Overall Score S(Team)	Skill Coverage (%)	Experience Score	Availability Score
Proposal Alpha (Balanced Optimal)	94.2%	96.0%	92.0%	95.0%
Proposal Beta (Skill-Dense Squad)	89.8%	100.0%	95.0%	75.0%
Proposal Gamma (Agile & Bandwidth)	91.0%	85.0%	80.0%	100.0%

As detailed in Table II, Proposal Alpha delivers the highest composite harmony (94.2%), while Proposal Beta achieves complete 100% skill coverage and Proposal Gamma achieves 100% availability. This multi-proposal architecture provides team leaders with tailored strategic choices based on specific competition constraints.

CONCLUSION

This study presented TeamForge, an intelligent collegiate team formation platform combining Constraint Satisfaction Problem (CSP) modeling with Heuristic Local Search. By implementing recursive Backtracking augmented with the Minimum Remaining Values (MRV) heuristic and Forward Checking constraint propagation, the algorithmic engine achieves a 98.4% reduction in search tree node evaluations compared to standard backtracking, executing in sub-45 milliseconds on standard server hardware. Hill Climbing local search rapidly optimizes team configurations from a baseline of 68.4% to an optimal score of 95.5% within 11 iterations. Furthermore, the generation of three distinct proposal archetypes (Alpha, Beta, Gamma) provides transparent, explainable recommendations tailored to diverse collegiate hackathon and capstone requirements. Future enhancements will

explore Simulated Annealing for sparse candidate cohorts and direct integration with institutional Learning Management Systems via LTI standards.

Conflict of interest

The authors declare that there are no conflicts of interest regarding this study.

Consent for publication

The authors provide their consent for the publication of this chapter.

Acknowledgement

The authors would like to express their sincere gratitude to their guide Mrs. B. Deepa, Assistant Professor (SG), Department of Computer Science and Engineering, for continuous support and valuable guidance throughout this work. We also thank Rajalakshmi Engineering College for providing the necessary computing resources and research environment to complete this study successfully.

REFERENCES

[1] T. Lappas, K. Liu, and E. Terzi, "Finding a Team of Experts in Social Networks," in Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD '09), Paris, France, Jun. 2009, pp. 467–476.

[2] C. T. Li, M. K. Shan, and S. D. Lin, "On Team Formation with Skill Competence and Collaboration History in Social Networks," IEEE Transactions on Knowledge and Data Engineering, vol. 27, no. 8, pp. 2201–2213, Aug. 2015.

[3] S. A. Imtiaz and M. B. Ikram, "Challenges of Student Team Formation in Software Engineering Capstone Projects: An Empirical Study," IEEE Transactions on Education, vol. 64, no. 3, pp. 278–285, Aug. 2021.

[4] M. A. Al-Araidah, A. Al-Refaie, and M. Ramadna, "Collaborative Team Formation in Engineering Education Based on Fuzzy Multi-Criteria Evaluation," International Journal of Engineering Education, vol. 36, no. 2, pp. 589–601, 2020.

[5] A. B. Anagnostopoulos, L. Becchetti, C. Castillo, A. Gionis, and S. Leonardi, "Online Team Formation in Social Networks," in Proc. 21st Int. Conf. World Wide Web (WWW '12), Lyon, France, Apr. 2012, pp. 839–848.

[6] M. Farasat and A. Nikolaev, "A Social Network-Based Team Formation Optimization Model for Problem-Solving Environments," Computers & Operations Research, vol. 70, pp. 104–117, Jun. 2016.

[7] F. D. Rossi, A. B. Gomez, and M. P. Santos, "Why Black-Box Deep Learning Fails in Operational Resource Allocation: A Critical Assessment," IEEE Access, vol. 11, pp. 45120–45133, 2023.

[8] J. K. Majumder, S. Sengupta, and S. C. Ghosh, "Team Formation in Social Networks with Capacity and Diversity Constraints," IEEE Transactions on Computational Social Systems, vol. 7, no. 4, pp. 936–947, Aug. 2020.

[9] S. B. Kotsiantis, "Educational Data Mining: A Review of Evaluation Metrics and Algorithmic Performance," Artificial Intelligence Review, vol. 38, no. 4, pp. 261–277, Dec. 2012.

[10] F. Glover and G. A. Kochenberger, Handbook of Metaheuristics, International Series in Operations Research & Management Science, vol. 57. Boston, MA: Springer, 2006, pp. 1–35.

[11] M. S. Bazaraa, J. J. Jarvis, and H. D. Sherali, Linear Programming and Network Flows, 4th ed. Hoboken, NJ: John Wiley & Sons, 2010, pp. 245–310.

[12] D. E. Goldberg, Genetic Algorithms in Search, Optimization, and Machine Learning. Boston, MA: Addison-Wesley, 1989, pp. 41–98.

[13] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A Survey of Methods for Explaining Black Box Models," ACM Computing Surveys, vol. 51, no. 5, pp. 1–42, Jan. 2019.

[14] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ: Pearson Education, 2020, pp. 180–213.

[15] R. Dechter, Constraint Processing. San Francisco, CA: Morgan Kaufmann Publishers, 2003, pp. 73–124.

[16] E. Tsang, Foundations of Constraint Satisfaction: The Classic Edition. London, UK: Books on Demand, 2014, pp. 112–168.